STUDENT PROJECT · SOLUTION DESCRIPTION

Quantum-Assisted Optimisation of Vehicle Test Configurations

Smit Babariya · 2021

Goal	Choose feature configurations for a small fleet of test vehicles so that every car is buildable and as many (weighted) test requirements as possible are covered, then schedule the resulting tests.
Approach	Formulation as SAT → weighted partial Max-SAT → job-shop scheduling; classical preprocessing combined with gate-based (QAOA) and annealing-based quantum methods.
Tools & methods	Python · PySAT · Tseitin transformation · PennyLane · QAOA · Quantum annealing · Max-SAT · Job-shop scheduling

1 Problem Statement

Before a new vehicle series goes into production, a limited number of test cars must be configured so that all planned tests can be carried out. Each car has to be a *valid* configuration (buildability), and each test needs at least one car that carries the relevant combination of features.

Buildability constraints

- There are n vehicles and m features that can be fitted to each vehicle.
- A binary variable b_{ij} indicates whether vehicle v_i has feature a_j.
- There are p configuration rules; each rule ϕ_k is a Boolean expression over the m feature variables and must hold for every vehicle:

$$\phi_k(b_{i1}, \ldots, b_{im}) = 1, \qquad i = 1, \ldots, n, \quad k = 1, \ldots, p \tag{1}$$

Equivalently, the whole fleet is buildable if

$$\bigwedge_{i=1}^{n} \bigwedge_{k=1}^{p} \phi_k(b_{i1}, \ldots, b_{im}) = 1. \tag{2}$$

Testing constraints

- Each test l requires a vehicle whose features satisfy a Boolean expression ψ_l:

$$\psi_l(b_{i1}, \ldots, b_{im}) = 1, \qquad l = 1, \ldots, q \tag{3}$$

- All cars must satisfy the buildability constraints, but it is sufficient for *at least one* car to satisfy each test requirement.

Notation

Symbol	Meaning
n	Number of test vehicles
m	Number of available features
v_i	Vehicle i
a_j	Feature j
b_{ij}	Binary variable: vehicle v_i has feature a_j
p	Number of configuration (buildability) constraints
$\phi_k(b_{i1}, \ldots, b_{im})$	Configuration constraint k for vehicle i, a Boolean expression of features
q	Number of test constraints
$\psi_l(b_{i1}, \ldots, b_{im})$	Test constraint l for vehicle i, a Boolean expression of features
w_l	Weight (priority) of test l, $w_l > 0$

2 Solution Approach

2.1 Satisfiability (SAT) formulation

The decision version asks: is there a feature configuration for the n cars such that all buildability constraints (1)–(2) hold and every test requirement (3) is met by at least one car?

$$\left(\bigwedge_{k=1}^{p} \bigwedge_{i=1}^{n} \phi_k(b_{i1}, \ldots, b_{im}) \right) \wedge \left(\bigwedge_{l=1}^{q} \bigvee_{i=1}^{n} \psi_l(b_{i1}, \ldots, b_{im}) \right) = 1 \tag{4}$$

In practice, some tests may have to be performed on more than one vehicle, e.g. on d vehicles; the formulation extends naturally to this case.

2.2 Weighted partial Max-SAT

Because not every test can necessarily be covered with a limited fleet, the problem is relaxed to an optimisation problem: find the feature assignment for the n test vehicles, each complying with the configuration rules, that maximises the weighted number of fulfilled test requirements:

$$\max_{b_{ij}} \; f(b_{11}, \ldots, b_{nm}) = \sum_{l=1}^{q} w_l \cdot \bigvee_{i=1}^{n} \psi_l(b_{i1}, \ldots, b_{im}) \tag{5}$$

subject to the configuration constraints (1). The weights $w_l > 0$ allow certain tests to be prioritised over others. Buildability rules act as *hard* clauses, test requirements as weighted *soft* clauses; the solution is the feature set with maximum benefit.

2.3 Test scheduling (job-shop problem)

- Once the vehicle configurations are fixed by the Max-SAT solution, the resulting tests must be scheduled.
- Each test l has a time window $[\, t_l^{\text{start}},\, t_l^{\text{end}} \,]$ in which it must be performed, for all $l = 1, \ldots, q$.
- Vehicles are modelled as jobs and tests as machines; scheduling is done on a day-to-day basis.

3 Quantum Solution Pipeline

1 Classical preprocessing

PySAT is used to simplify the formula and bring it into CNF via the Tseitin transformation, reducing it to a size that current quantum hardware can handle.

→

2 QAOA for Max-SAT

The weighted Max-SAT instance is encoded as a cost Hamiltonian and solved with the Quantum Approximate Optimisation Algorithm, using a PennyLane feedback (variational) loop.

→

3 Annealing for scheduling

The job-shop scheduling problem is formulated for a quantum annealer, which returns the start time of every test on every vehicle.

Evaluation target. The main criteria for judging the approach are its innovative character and how efficiently it scales, i.e. favouring solutions with lower hardware requirements.

4 Worked Example

Feature set

Feature	Meaning
a_1	Small engine
a_2	Big engine
a_3	Strong subwoofer
a_4	Seat heating
a_5	Comfort seating

Test constraints

Test	Required features
Brakes 1	$\psi = b_{i1}$
Brakes 2	$\psi = b_{i2}$
Noise level	$\psi = b_{i2} \wedge b_{i3}$
Bass & seat interaction	$\psi = b_{i3} \wedge b_{i4}$
Power consumption	$\psi = b_{i2} \wedge b_{i3} \wedge b_{i5}$

Classical solution: features a_2 (big engine) and a_3 (strong subwoofer).

Quantum solution (Max-SAT)

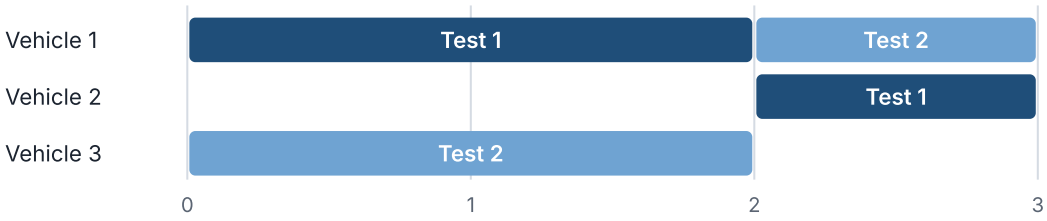
```
$ python test.py
c Number of variables occuring in the formula: 4 max variable = 5 → remapping
-0.000000 iter=6 maxdiff=0.000001
c first lower bound: 0
o 0
c 1 branches 3 propagates
c total generalized unit propagation = 4, success = 100.00%
s OPTIMUM FOUND
c Optimal Solution = 0
v -2 1 4 -3 5
[-1, 1, 1, -1]
```

Solver output: an optimal assignment was found; three cars satisfy their buildability constraints with this configuration.

Scheduling solution

```
$ python job-shop-scheduling
Jobs and their machine-specific tasks:
vehicle1 : [('Test1', 2), ('Test2', 1)]
vehicle2 : [('Test1', 1)]
vehicle3 : [('Test2', 2)]

Jobs and the start times of each task:
vehicle1 : [0, 2]
vehicle2 : [2]
vehicle3 : [0]
```



Resulting schedule (time units). Each test rig is used by only one vehicle at a time, and all tests finish by $t = 3$.